

Multi User With Osc Groups Setup

Contents

1	One client per workstation	2
2	Local monitoring does not depend on an OSCGroups echo	2
3	Signal path on every workstation	3
4	Complete workstation pipelines	3
4.1	Workstation A	3
4.2	Workstation B	3
5	Shared and unique settings	3
6	Port map on each workstation	6
7	Before the rehearsal	6
8	Startup procedure on every workstation	7
8.1	1. Start Godot	7
8.2	2. Start one shared local decoder	7
8.2.1	Basic version	7
8.2.2	Advanced version	7
8.3	3. Configure Bmc for the local and remote performers	7
8.4	4. Start one OscGroupClient	8
8.5	5. Start the local encoder	8
8.6	6. Enable XR-Animator output	9
9	Verify the collaboration	9
10	Avoid feedback and identity errors	9
10.1	1. Give every transmitting encoder a unique identity	9
10.2	2. Send Bmc output only to the local decoder	10
10.3	3. Feed OSCGroups only from BunrakuOSCEncoder	10

10.4 4. Keep the <code>OscGroupClient</code> receive and transmit paths separate	11
10.5 5. Use one unique <code>OscGroupClient</code> username per client	12
10.6 6. Do not expect a server echo	12
11 Diagnose ports	12
12 Shutdown on every workstation	12
13 Related detail	12

This is the canonical setup for collaborating between two or more workstations over the Internet. Each workstation runs the same local applications and exactly one full-duplex `OscGroupClient`.

1 One client per workstation

One `OscGroupClient` is enough because it handles both network directions. The encoder separately gives Bmc its local copy:

```
local encoder -> local Bmc
local encoder -> client localTxPort -> OSCGroups network
OSCGroups network -> client localRxPort -> local Bmc
```

Do not start separate sending and receiving clients on an ordinary workstation. Two clients are needed only when deliberately simulating two workstations on one computer, joining two different groups, or using two independent `OSCGroups` identities.

2 Local monitoring does not depend on an `OSCGroups` echo

The local encoder's network packets are sent to the *other* clients in the group. They are not returned to the originating client's `localRxPort`. This is the normal `OSCGroups` behavior, not a fault or missing option.

`BunrakuOSCEncoder` solves this by sending every completed frame to two local destinations by default:

- Bmc on 127.0.0.1:57130;
- `OscGroupClient` on 127.0.0.1:22244.

Consequently, on Workstation A:

```

+--> A's Bmc
PerformerA -> A's encoder
+--> A's OscGroupClient -> Workstations B, C, ...
```

```
Workstations B, C, ... -> A's OscGroupClient -> A's Bmc
```

A's Bmc receives `PerformerA` directly from A's encoder and receives the remote performers through `OscGroupClient`. Thus the local Godot scene can render both local and remote motion without a server echo. Do not start a second client merely to manufacture an echo.

3 Signal path on every workstation

```
local XR-Animator
-> VMC 127.0.0.1:39537
local BunrakuOSCEncoder
-> local copy 127.0.0.1:57130 (local Bmc)
-> network copy 127.0.0.1:22244 (OscGroupClient)
one local OscGroupClient
-> OSCGroups server and remote collaborators

remote Bunraku frames
-> same local OscGroupClient
-> 127.0.0.1:57130
local Bmc in SuperCollider
-> routed Bunraku frames 127.0.0.1:39538
one local BunrakuOSCDecoder
-> local Godot VMC ports 39539, 39540, ...
```

Each workstation may render its own performer and any remote performers it chooses to instantiate in its local Godot scene. Its own stream reaches Bmc directly from the encoder, not through the client's receive path.

4 Complete workstation pipelines

The following two diagrams show a concrete two-workstation example. **PerformerA** and **PerformerB** are stable avatar identities carried by the route-free frames; **workstation-a-xr-animator** and **workstation-b-xr-animator** are unique source identities. The avatar names shown inside the Godot boxes are the visual models driven by those streams on each workstation.

4.1 Workstation A

Workstation A renders its local **PerformerA** stream as Ishidomaru on port 39539 and the remote **PerformerB** stream as Mother on port 39540.

4.2 Workstation B

Workstation B renders its local **PerformerB** stream as Mother on port 39539 and the remote **PerformerA** stream as Ishidomaru on port 39540.

5 Shared and unique settings

All collaborators must agree on:

- OSCGroups server address;
- OSCGroups server port, normally 22242;
- group name;
- group password;

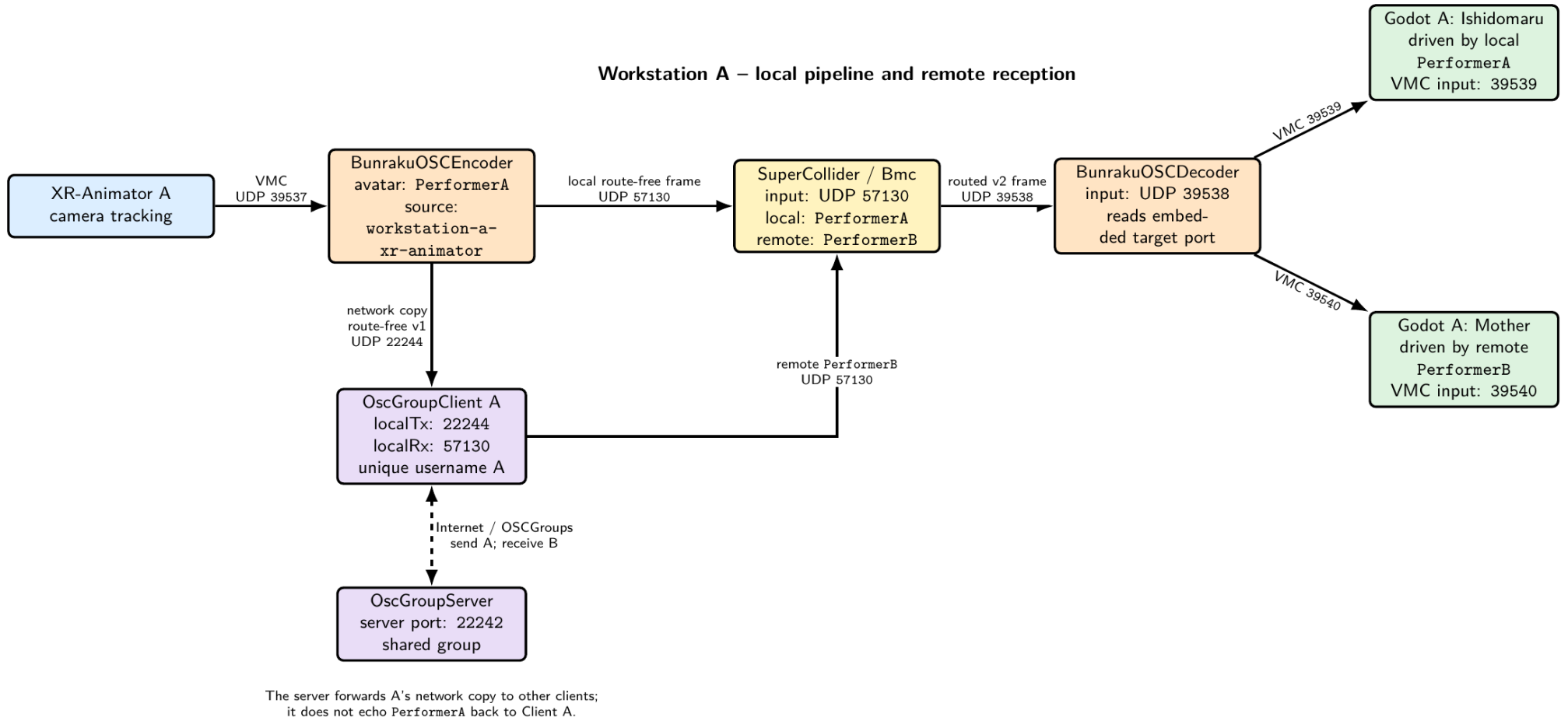


Figure 1: Workstation A: local XR-Animator path, OSCGroups exchange, SuperCollider routing, decoding, and Godot destinations.

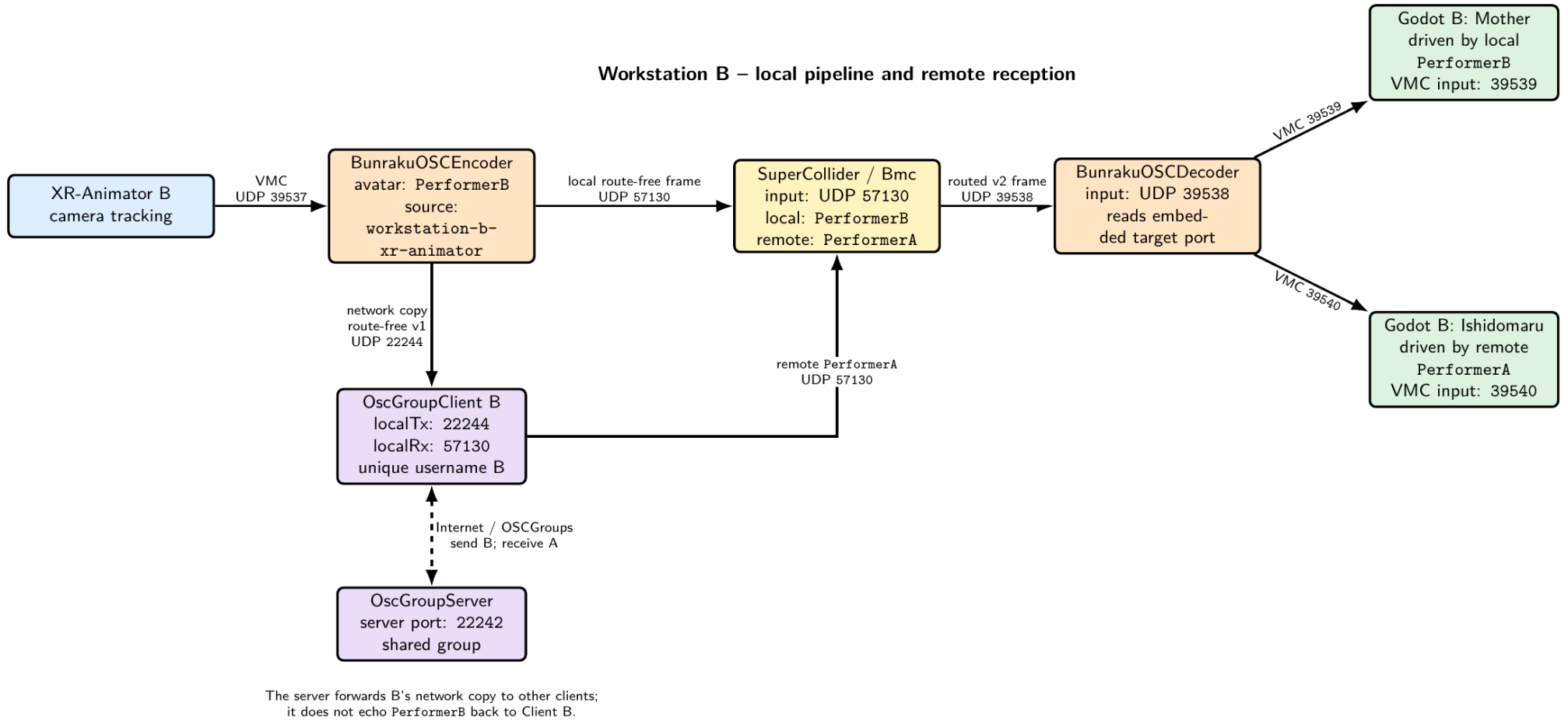


Figure 2: Workstation B: local XR-Animator path, OSCGroups exchange, SuperCollider routing, decoding, and Godot destinations.

- stable avatar/source names for each performer.

Every workstation must have its own:

- OSCGroups username;
- OSCGroups user password;
- `localToRemotePort` when required by the network/NAT;
- encoder `--avatar` name, such as `PerformerA` or `PerformerB`.

Local application ports may use the same numbers on different workstations because each computer has its own network stack.

6 Port map on each workstation

Port	Listener	Sender
39537	local <code>BunrakuOSCEncoder</code>	local XR-Animator
22244	local <code>OscGroupClient</code> tx input	local encoder
57130	local <code>Bmc/SuperCollider</code>	local encoder and <code>OscGroupClient</code> rx output
39538	local shared decoder	local <code>Bmc</code> avatars
39539	first local Godot VMC receiver	local decoder
39540	second local Godot VMC receiver	local decoder

`localToRemotePort` is an additional client port. Choose a free value distinct from 22244 and 57130; collaborators behind the same NAT may need different values.

7 Before the rehearsal

Exchange these values privately:

```
SERVER_ADDRESS
GROUP_NAME
GROUP_PASSWORD
each workstation's unique USER_NAME
each performer's stable AVATAR_NAME
```

Do not store real passwords in this repository. Decide which remote performer is rendered by which local Godot avatar and VMC port.

Example assignment on Workstation B:

Stream	Local rendered avatar	Godot VMC port
PerformerB (local, direct)	Mother	39539
PerformerA (remote, network)	Ishidomaru	39540

8 Startup procedure on every workstation

Perform these steps in order. Commands containing uppercase names are templates: replace them with the agreed values.

8.1 1. Start Godot

Open and run the project containing the local avatars. Give every independently animated avatar a distinct VMC port and distinct body/face tracker names.

For the example above:

- Mother listens on 39539;
- Ishidomaru listens on 39540.

8.2 2. Start one shared local decoder

8.2.1 Basic version

Use this for simplicity. SuperCollider must embed the correct destination port in every protocol-version-2 frame, either from the active session definition or from manually configured Bmc avatars.

```
BunrakuOSCDecoder \  
  --listen-port 39538 \  
  --verbose
```

With no allow-list, the decoder accepts any valid destination port embedded in a protocol-version-2 frame. Protocol-version-1 frames are still rejected unless a fallback `--target-port` is supplied.

8.2.2 Advanced version

Use this if you want to make sure that only the ports that you allow are used.

```
BunrakuOSCDecoder \  
  --listen-port 39538 \  
  --allow-target-port 39539 \  
  --allow-target-port 39540 \  
  --verbose
```

The decoder reads the destination from each frame as before, but rejects frames whose embedded port is not 39539 or 39540.

Keep the Terminal open.

8.3 3. Configure Bmc for the local and remote performers

The Bmc avatar ID must match the `--avatar` name used by the remote performer's encoder. The display name may match the local Godot character.

Example on Workstation B:

```
(  
Bmc.reset;
```

```

~decoder = NetAddr("127.0.0.1", 39538);

// Local PerformerB drives local Mother through the encoder's direct copy.
Bmc.addAvatar(\PerformerB, "Mother");
Bmc.avatar(\PerformerB).output_(~decoder);
Bmc.avatar(\PerformerB).vmcPort_(39539);

// Remote PerformerA drives local Ishidomaru through OscGroupClient.
Bmc.addAvatar(\PerformerA, "Ishidomaru");
Bmc.avatar(\PerformerA).output_(~decoder);
Bmc.avatar(\PerformerA).vmcPort_(39540);

Bmc.start(57130);
Bmc.status;
)

```

Register only the local and remote performers that this workstation needs to render. The posted Bmc status must include **running: true** and **port: 57130**.

8.4 4. Start one OscGroupClient

From the top-level 260715_ICLC27 project directory:

```

AppsAndCode/OSCGroups/bin/macos/OscGroupClient \
  SERVER_ADDRESS \
  22242 \
  LOCAL_TO_REMOTE_PORT \
  22244 \
  57130 \
  USER_NAME \
  USER_PASSWORD \
  GROUP_NAME \
  GROUP_PASSWORD

```

The client should report successful registration with the server and group. Keep this one client running for both sending and receiving.

8.5 5. Start the local encoder

Choose the unique avatar name assigned to this workstation. Workstation A might use:

```

BunrakuOSCEncoder \
  --avatar "PerformerA" \
  --source "workstation-a-xr-animator" \
  --verbose

```

The default ports already send one copy to Bmc 57130 and another to OscGroupClient 22244. Workstation B must use a different stable name, for example PerformerB. Keep the encoder Terminal open.

8.6 6. Enable XR-Animator output

On every workstation that contributes live motion, configure XR-Animator:

Host: 127.0.0.1

Port: 39537

Enable VMC output.

9 Verify the collaboration

Check the pipeline from left to right:

1. Each transmitting encoder reports increasing `received`, `sent`, `bmc_sent`, and `oscggroups_sent` counts.
2. Each `OscGroupClient` reports successful group registration.
3. On each workstation, `Bmc.status` receives its local frames directly and remote frames through `OSCGroups`.
4. The decoder reports increasing `received` and `sent` counts and the expected target ports.
5. Each remote performer moves the intended local Godot avatar.

For a minimal two-person test:

- A transmits `PerformerA`; B registers and renders `PerformerA`.
- B transmits `PerformerB`; A registers and renders `PerformerB`.

10 Avoid feedback and identity errors

10.1 1. Give every transmitting encoder a unique identity

The encoder's `--avatar` identifies the motion stream that `Bmc` must select. Its `--source` identifies the particular performer/workstation/capture source. Use stable, unique values on every transmitting workstation.

Correct examples:

```
# Workstation A
BunrakuOSCEncoder \
  --avatar "PerformerA" \
  --source "workstation-a-xr-animator"
```

```
# Workstation B
BunrakuOSCEncoder \
  --avatar "PerformerB" \
  --source "workstation-b-xr-animator"
```

Do *not* launch both encoders with the same identity:

```
# WRONG on Workstation A
BunrakuOSCEncoder --avatar "Performer" --source "xr-animator"
```

```
# WRONG on Workstation B: indistinguishable from A
BunrakuOSCEncoder --avatar "Performer" --source "xr-animator"
```

With the wrong example, Bmc cannot reliably distinguish which person produced a frame. Both streams may drive the same registered Bmc avatar, and source-filtered recording cannot separate them.

The Bmc avatar ID on a receiving workstation must match the remote encoder's `--avatar` value. For example:

```
// Receives frames whose encoder used --avatar "PerformerA".
Bmc.addAvatar(\PerformerA, "Ishidomaru");
```

10.2 2. Send Bmc output only to the local decoder

Bmc adds the final local Godot VMC port to a protocol-version-2 frame. That routed frame belongs on the local decoder input, normally 39538:

```
// CORRECT
~decoder = NetAddr("127.0.0.1", 39538);
Bmc.avatar(\PerformerA).output_(~decoder);
Bmc.avatar(\PerformerA).vmcPort_(39539);
```

Do *not* point a Bmc avatar at `OscGroupClient`'s transmission port:

```
// WRONG: retransmits Bmc's locally routed result to the group.
Bmc.avatar(\PerformerA).output_(NetAddr("127.0.0.1", 22244));
```

The embedded 39539 in that frame is meaningful only for this workstation's Godot scene. Another workstation may assign the same performer to a different avatar and port. Sending the routed result back through OSCGroups therefore distributes local routing information and can also create repeated circulation when another workstation makes the same mistake.

10.3 3. Feed OSCGroups only from BunrakuOSCEncoder

The encoder produces route-free protocol-version-1 frames. These are the portable motion-source messages intended for other workstations. Its default network output is already correct:

```
# CORRECT: defaults to Bmc 57130 and OscGroupClient 22244.
BunrakuOSCEncoder \
  --avatar "PerformerA" \
  --source "workstation-a-xr-animator"
```

Do not start another Python converter between Bmc and `OscGroupClient`. In particular, do not use `BunrakuOSCDecoder` as a network sender:

```
# WRONG architecture: the decoder reconstructs VMC for Godot; it does not
# create the portable Bunraku source frames that should enter OSCGroups.
BunrakuOSCDecoder \
  --listen-port 39538 \
  --target-port 22244
```

For a route-free version-1 input, that wrong command sends reconstructed VMC—not a Bunraku frame—to `OscGroupClient`. For a routed version-2 input, the embedded destination overrides the fallback `--target-port`, so this command does not even redirect it to 22244. In neither case is the decoder the correct network transmitter.

Also do not change the encoder's OSCGroups destination to Bmc's port:

```
# WRONG: both encoder outputs now lead to 57130, so nothing reaches OSCGroups.
BunrakuOSCEncoder \
  --oscggroups-port 57130 \
  --avatar "PerformerA" \
  --source "workstation-a-xr-animator"
```

Leave the encoder's `--oscggroups-port` at 22244 unless the client's `localTxPort` was deliberately changed to the same new value.

10.4 4. Keep the `OscGroupClient` receive and transmit paths separate

The client arguments must retain these two distinct roles:

```
localTxPort = 22244    local encoder -> OSCGroups
localRxPort = 57130    OSCGroups -> local Bmc
```

This is the correct positional fragment:

```
OscGroupClient \
  SERVER_ADDRESS 22242 LOCAL_TO_REMOTE_PORT \
  22244 57130 \
  USER_NAME USER_PASSWORD GROUP_NAME GROUP_PASSWORD
```

Do *not* set `localRxPort` to the same port as `localTxPort`:

```
# WRONG: received group traffic is delivered to the client's own transmit input.
OscGroupClient \
  SERVER_ADDRESS 22242 LOCAL_TO_REMOTE_PORT \
  22244 22244 \
  USER_NAME USER_PASSWORD GROUP_NAME GROUP_PASSWORD
```

Do not create a SuperCollider relay from Bmc's input back to the client either:

```
// WRONG: every remote frame received on 57130 is transmitted to the group again.
OSCdef(\badNetworkLoop, { |msg|
  NetAddr("127.0.0.1", 22244).sendMsg(*msg)
}, '/bunraku/vmc/frame', recvPort: 57130);
```

If this test code has ever been evaluated, remove it:

```
OSCdef(\badNetworkLoop).free;
```

Such a return path can make workstations repeatedly retransmit one another's packets, producing duplicated motion, rising traffic, and unstable playback.

10.5 5. Use one unique `OscGroupClient` username per client

Do not run two clients with the same username. A username identifies one client to the OSCGroups server; duplicate use may replace or confuse the registered endpoint.

10.6 6. Do not expect a server echo

The group does not return a workstation's own frames for local monitoring. The encoder's direct copy to Bmc port 57130 provides the local path instead.

11 Diagnose ports

On each Mac:

```
lsof -nP -iUDP:39537
lsof -nP -iUDP:22244
lsof -nP -iUDP:57130
lsof -nP -iUDP:39538
```

Only one local process should listen on each port. If a client cannot register, verify the server address, group credentials, user credentials, firewall, and Internet connection.

12 Shutdown on every workstation

1. Disable XR-Animator VMC output.
2. Evaluate `Bmc.stop` in SuperCollider.
3. Press `Control-C` in the encoder Terminal.
4. Press `Control-C` in the single `OscGroupClient` Terminal.
5. Press `Control-C` in the decoder Terminal.
6. Stop the running Godot project.

13 Related detail

- [OscGroupClient Use and Configuration](#)
- [OSC Encoder and Decoder Use and Configuration](#)
- [Avatar Port Numbers](#)
- [Port Number Specification](#)
- [Troubleshooting Port Numbers](#)